

**SIMATS ENGINEERING
ASSESSMENT TOOL 1**

COURSE CODE: ITA 06

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO3: Bayes Theorem – Concept Learning – Maximum Likelihood – Minimum Description Length Principle – Bayes Optimal Classifier – Gibbs Algorithm – Naïve Bayes Classifier – Bayesian Belief Network – EM Algorithm – Probability Learning – Sample Complexity – Finite and Infinite Hypothesis Spaces – Mistake Bound Model, (BL 6)

Assessment Tool Weightage: 40%

QUESTIONS: 15

Total Marks: 25

**Annexure A
Experiments**

Code of Conduct:

I V.Madhulika (192312620) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

V.Madhulika

Signature of the student

1. Design and conduct an experiment to implement a Naïve Bayes classifier on a real-world dataset. Train the model using different feature sets and evaluate its performance using accuracy, precision, and recall. Analyze how assumptions of feature independence affect the results. Compare outcomes with and without Laplace smoothing, and interpret how probability estimation impacts classification performance.

Answer:

Objective:

To implement and evaluate a Naïve Bayes classifier on a real-world dataset (e.g., the SMS Spam Collection dataset) under different feature sets and smoothing configurations.

Procedure:

- Load and preprocess the dataset (tokenize text, remove stop words, convert to Bag-of-Words / TF-IDF feature vectors).
- Split the data into training (80%) and testing (20%) sets.
- Train Multinomial Naïve Bayes on (i) the full feature set and (ii) a reduced set of the most informative words.
- Evaluate each model using accuracy, precision, and recall on the test set.
- Repeat training with Laplace (add-1) smoothing disabled ($\alpha = 0$) and enabled ($\alpha = 1$) and compare results.

Observations & Analysis:

The full feature set generally yields higher recall and accuracy than the reduced set, at the cost of more computation. Without smoothing, any test word never seen with a given class produces a zero probability, which forces the entire posterior for that class to zero and causes misclassification — precision and recall drop sharply for such cases. With Laplace smoothing, all probability estimates remain non-zero, so the classifier handles unseen word/class combinations gracefully, and performance becomes noticeably more stable across feature sets. Although the conditional-independence assumption between words does not perfectly hold in real text, the classifier remains robust because classification depends on the relative ranking of class probabilities rather than their exact values.

Conclusion:

Laplace smoothing and informed feature selection both meaningfully improve Naïve Bayes robustness and accuracy, confirming that the quality of probability estimation directly affects classification performance even when the independence assumption is only approximately satisfied.

2. Perform an experiment to estimate parameters of a probability distribution using Maximum Likelihood Estimation (MLE) and compare it with a Bayesian estimation approach. Use a dataset to compute parameters such as mean and variance, and observe how prior assumptions influence results. Analyze differences in estimation under small and large sample sizes, and interpret the impact on model reliability.

Answer:

Objective:

To estimate the mean and variance of a normally distributed dataset using MLE and compare the results with Bayesian estimation using an informative prior.

Procedure:

- Generate/collect samples from a distribution with a known true mean and variance.
- Compute the MLE estimates: sample mean and sample variance.
- Compute the Bayesian posterior mean/variance using a Normal prior with an assumed prior mean and variance, updated with the observed data.
- Repeat the comparison for a small sample ($n = 10$) and a large sample ($n = 1000$).

Observations & Analysis:

For the large sample ($n = 1000$), the MLE and Bayesian estimates converge closely to each other and to the true parameters, since the abundant data dominates the influence of the prior. For the small sample ($n = 10$), the MLE estimate is noisy and can deviate substantially from the true value, whereas the Bayesian estimate is pulled toward the prior mean, producing a more stable and regularized estimate even though it introduces some bias if the prior is inaccurate.

Conclusion:

Prior assumptions exert a strong, useful regularizing effect when the sample size is small, improving reliability; as sample size increases, the influence of the prior diminishes and both approaches converge — showing Bayesian estimation is particularly valuable in low-data regimes.

3. Implement the Expectation-Maximization (EM) algorithm for a mixture model and conduct experiments to study its convergence behavior. Vary initialization parameters and number of clusters, and observe their effect on convergence speed and final likelihood values. Record and analyze cases where the algorithm converges to local optima. Suggest strategies to improve convergence stability and performance.

Answer:**Objective:**

To implement EM for a Gaussian Mixture Model (GMM) and study how initialization and the number of components (K) affect convergence.

Procedure:

- Initialize GMM parameters (means, covariances, mixing weights) randomly.
- Iterate the E-step (compute cluster-membership responsibilities) and M-step (update parameters) until the change in log-likelihood falls below a threshold.
- Repeat with several random initializations and with $K = 2, 3, 4, 5$ clusters.
- Record the number of iterations to convergence and the final log-likelihood for each run.

Observations & Analysis:

Convergence speed and final log-likelihood vary noticeably with initialization: poor initial means can lead to slow convergence or to a lower final log-likelihood than other runs — a clear sign of convergence to a local optimum. Increasing K generally raises the achievable likelihood but risks overfitting, empty clusters, or singular covariance matrices.

Strategies to Improve Stability:

- Use K-Means++ style initialization for the initial means instead of pure random placement.
- Run multiple random restarts and retain the solution with the highest log-likelihood.
- Add a small regularization term to covariance matrices to avoid degenerate/singular solutions.

4. Construct a Bayesian Belief Network for a given problem domain and perform inference under different evidence conditions. Conduct experiments by modifying conditional probabilities and

observing changes in posterior probabilities. Analyze how dependencies between variables influence the results. Evaluate the robustness of the model and interpret how uncertainty is handled in probabilistic reasoning.

Answer:

Objective:

To construct a Bayesian Belief Network (e.g., the classic Burglary–Earthquake–Alarm–JohnCalls–MaryCalls domain) and study inference under varying evidence.

Procedure:

- Define the nodes and their conditional probability tables (CPTs) based on domain knowledge.
- Use exact inference (e.g., variable elimination via a library such as pgmpy) to compute posterior probabilities, e.g., $P(\text{Burglary} \mid \text{JohnCalls} = \text{True})$.
- Vary the evidence combination (single vs. multiple calls) and re-run inference.
- Modify individual CPT entries (e.g., increase $P(\text{Alarm} \mid \text{Burglary})$) and observe how downstream posteriors shift.

Observations & Analysis:

The posterior probability of Burglary increases substantially when both JohnCalls and MaryCalls are True compared to only one call being observed, since corroborating evidence reinforces the belief. Changing a CPT entry (e.g., raising $P(\text{Alarm} \mid \text{Burglary})$) directly shifts all downstream posteriors that depend on Alarm. Dependencies encoded by the network edges mean evidence on a child node propagates backward to update beliefs about its parents (diagnostic reasoning), while evidence on a parent propagates forward to its children (causal reasoning).

Conclusion:

The network remains robust as long as CPTs are reasonably accurate, and it explicitly represents uncertainty as probability distributions rather than fixed values — making probabilistic reasoning transparent, adjustable, and interpretable.

5. Implement a concept learning algorithm (such as Candidate Elimination) and perform experiments to analyze the effect of hypothesis space size on learning performance. Vary the number of attributes and training examples, and observe changes in sample complexity and convergence. Analyze how finite and infinite hypothesis spaces influence generalization ability and learning efficiency, and interpret results using the mistake bound model.

Answer:

Objective:

To implement the Candidate-Elimination algorithm and study how hypothesis space size (number of attributes) and number of training examples affect convergence.

Procedure:

- Represent hypotheses as conjunctions of attribute constraints; initialize the Specific boundary S and General boundary G .
- Process each training example, updating S and G according to the Candidate-Elimination rules.
- Repeat with varying numbers of attributes (e.g., 2, 4, 6) and varying numbers of training examples (e.g., 4, 8, 16).

Observations & Analysis:

More attributes exponentially increase hypothesis space size, requiring more training examples for S and G to converge to a single consistent hypothesis (higher sample complexity). With too few examples, the version space (gap between S and G) remains large, indicating unresolved ambiguity. Finite hypothesis spaces guarantee eventual convergence given enough examples and bound the number of mistakes an online learner makes (mistake bound on the order of $\log_2 |H|$); infinite hypothesis spaces (e.g., continuous-valued attributes) do not guarantee convergence unless the space has bounded complexity (e.g., bounded VC-dimension).

Conclusion:

Sample complexity grows with hypothesis space size, so controlling hypothesis space size through appropriate inductive bias is essential for efficient generalization from limited training data.

6. Design and conduct an experiment to implement a Decision Tree classifier on a real-world dataset. Train the model using different splitting criteria such as entropy and Gini index, and evaluate performance using accuracy and F1-score. Analyze the impact of tree depth and pruning on overfitting, and compare results across different configurations.

Answer:

Objective:

To train Decision Tree classifiers with entropy and Gini index criteria and study the effect of tree depth and pruning on overfitting.

Procedure:

- Split the dataset into training and test sets.
- Train trees with criterion = 'entropy' and criterion = 'gini' at several max_depth values (3, 5, 10, unrestricted).
- Evaluate accuracy and F1-score on both the training and test sets for each configuration.
- Apply cost-complexity pruning to the unrestricted tree and compare test performance before and after pruning.

Observations & Analysis:

Both criteria typically produce similar overall results. Unrestricted-depth trees reach near-100% training accuracy but noticeably lower test accuracy — a clear sign of overfitting. Shallow trees (depth 3) underfit slightly but generalize more consistently. Pruning the unrestricted tree measurably narrows the train–test accuracy gap, improving test F1-score.

Conclusion:

Controlling tree depth and applying pruning are effective, complementary techniques for balancing bias and variance; the choice between entropy and Gini index has a comparatively smaller effect on performance than depth control.

7. Perform an experiment using the K-Nearest Neighbors (KNN) algorithm on a classification dataset. Vary the value of K and distance metrics (Euclidean, Manhattan) to observe their effect on model performance. Evaluate using accuracy and confusion matrix, and analyze how feature scaling influences the classification results and decision boundaries.

Answer:

Objective:

To evaluate KNN classifier performance across K values and distance metrics, with and without feature scaling.

Procedure:

- Split the dataset into training and test sets.
- Train/evaluate KNN for $K = 1, 3, 5, 7, 11$ using both Euclidean and Manhattan distance.
- Repeat the experiment on raw (unscaled) features and again after standardizing all features.
- Record accuracy and the confusion matrix for each configuration.

Observations & Analysis:

Very small K ($K = 1$) tends to overfit — high variance and sensitivity to noisy points; very large K oversmooths the decision boundary, increasing bias. An intermediate K (around 5–7) typically gives the best accuracy/confusion-matrix balance. The choice of distance metric affects the exact shape of the decision boundary but generally has a smaller effect than scaling. Without scaling, features with larger numeric ranges dominate the distance calculation, skewing neighbor selection; after standardization, accuracy improves and the decision boundaries become more balanced across all features.

Conclusion:

Feature scaling is essential for distance-based algorithms like KNN, and K should be tuned (e.g., via cross-validation) to balance bias and variance.

8. Implement a Support Vector Machine (SVM) model and conduct experiments using different kernel functions such as linear, polynomial, and RBF. Evaluate the model using accuracy and precision, and analyze how kernel choice and hyperparameters like C and gamma affect the decision boundary and classification performance.

Answer:**Objective:**

To train SVM classifiers with linear, polynomial, and RBF kernels and study the effect of kernel choice and the C/gamma hyperparameters.

Procedure:

- Split the dataset into training and test sets.
- Train SVMs with kernel = 'linear', 'poly' (degree 3), and 'rbf'.
- Vary C over {0.1, 1, 10, 100} and gamma over {'scale', 0.01, 0.1, 1} for the RBF kernel.
- Evaluate accuracy and precision on the test set, and visualize decision boundaries on a 2D projection.

Observations & Analysis:

The linear kernel performs well only when the data is close to linearly separable; the polynomial and RBF kernels capture non-linear boundaries, with RBF generally the most flexible. A small C produces a wider margin that tolerates more misclassification (higher bias, lower variance); a large C fits the training data tightly and risks overfitting. A small gamma gives smooth, simple boundaries; a large gamma creates tightly-fit, complex boundaries that are prone to overfitting.

Conclusion:

Kernel choice and hyperparameter tuning (via grid search / cross-validation) jointly determine an SVM's ability to balance fitting the training data and generalizing to new samples.

9. Design an experiment to implement a clustering algorithm such as K-Means on an unlabeled dataset. Vary the number of clusters and initialization methods, and evaluate performance using metrics like inertia and silhouette score. Analyze how cluster separation and feature scaling impact the quality and stability of clustering results.

Answer:

Objective:

To apply K-Means to an unlabeled dataset, varying K and initialization method, and to evaluate clustering quality using inertia and silhouette score.

Procedure:

- Standardize the dataset's features.
- Run K-Means for $K = 2$ to 8 using both random and K-Means++ initialization.
- Record inertia (within-cluster sum of squares) and silhouette score for each K, and plot the Elbow curve.

Observations & Analysis:

Inertia decreases monotonically as K increases, as expected; the 'elbow' point in the inertia curve together with the K achieving the highest silhouette score together indicate the most natural number of clusters. K-Means++ initialization produces more consistent, higher-silhouette results across repeated runs than random initialization, which occasionally converges to a poor local arrangement. Unscaled features with large numeric ranges distort distance calculations, reducing cluster separation quality and stability.

Conclusion:

Proper feature scaling and K-Means++ initialization substantially improve clustering quality and run-to-run stability; combining the Elbow method with silhouette score gives a more reliable choice of K than either alone.

10. Conduct an experiment using a logistic regression model for binary classification. Train the model with and without regularization (L1 and L2), and evaluate performance using accuracy, precision, and recall. Analyze the effect of regularization on model complexity, overfitting, and interpretability of learned coefficients.

Answer:

Objective:

To train logistic regression with no regularization, L1 regularization, and L2 regularization, and compare their effects.

Procedure:

- Split the dataset into training and test sets.
- Train logistic regression models with penalty = None, 'l1' (liblinear/saga solver), and 'l2', across a few regularization strengths (C).
- Evaluate accuracy, precision, and recall on the test set, and inspect the learned coefficients for each configuration.

Observations & Analysis:

Without regularization, coefficients can grow large, especially with correlated features, increasing the risk of overfitting on noisy data. L2 regularization shrinks all coefficients smoothly toward zero, reducing variance while retaining every feature in the model. L1 regularization drives some coefficients

exactly to zero, performing implicit feature selection and yielding a sparser, more interpretable model — sometimes at a small cost to raw accuracy.

Conclusion:

Regularization improves generalization and controls model complexity; L1 is preferable when interpretability and feature selection matter, while L2 is preferable when most features are believed relevant but need shrinkage.

11. Design and conduct an experiment to implement a Random Forest classifier on a real-world dataset. Train the model with varying numbers of trees and maximum depth. Evaluate performance using accuracy, precision, recall, and F1-score. Analyze how ensemble size and feature randomness affect overfitting and generalization, and compare results with a single decision tree.

Answer:

Objective:

To train Random Forest classifiers with varying number of trees and max depth, and compare results against a single decision tree.

Procedure:

- Split the dataset into training and test sets.
- Train Random Forest classifiers with `n_estimators` in {10, 50, 100, 300} and `max_depth` in {5, 10, unrestricted}.
- Evaluate accuracy, precision, recall, and F1-score on the test set for each configuration.
- Train one standalone, unrestricted-depth Decision Tree as a baseline for comparison.

Observations & Analysis:

Accuracy and F1-score improve and then stabilize as the number of trees increases, with diminishing returns beyond roughly 100–200 trees. Even with unrestricted depth, a large forest generalizes far better than a single unrestricted-depth decision tree, since averaging over many de-correlated trees (built via bootstrap sampling and random feature selection at each split) reduces variance. The single decision tree shows a much larger train–test performance gap, confirming heavier overfitting.

Conclusion:

Ensemble size and feature randomness are the key mechanisms by which Random Forest reduces overfitting compared to a single tree, at some added cost in training time and interpretability.

12. Perform an experiment using a Principal Component Analysis (PCA) technique for dimensionality reduction. Apply PCA on a high-dimensional dataset and vary the number of principal components retained. Evaluate the impact on model performance using accuracy and computational efficiency. Analyze how variance retention influences information loss and classification results.

Answer:

Objective:

To apply PCA to a high-dimensional dataset and study the effect of the number of retained components on downstream classification accuracy and training time.

Procedure:

- Standardize the dataset and fit PCA, inspecting the explained-variance ratio of each component.

- Train a classifier (e.g., Logistic Regression) on the original features and again on PCA-reduced features retaining approximately 95%, 90%, and 75% of the total variance.
- Compare test accuracy and training time across configurations.

Observations & Analysis:

Retaining components that explain roughly 90–95% of variance typically preserves most of the classification accuracy while substantially reducing dimensionality and training time. Retaining too few components (e.g., only 75% variance) causes noticeable accuracy loss due to discarded information. Retaining all components offers no dimensionality-reduction benefit.

Conclusion:

PCA provides a favorable trade-off between computational efficiency and information retention; the optimal number of components should be chosen from the cumulative explained-variance curve and validated against downstream task accuracy rather than fixed arbitrarily.

13. Implement a Gradient Descent optimization algorithm and conduct experiments by varying learning rates and initialization values. Observe convergence behavior and record the number of iterations required to reach optimal values. Analyze cases of slow convergence or divergence, and suggest improvements such as momentum or adaptive learning rate methods.

Answer:

Objective:

To implement batch gradient descent for a convex loss function (e.g., linear regression's mean squared error) and study the effect of learning rate and initialization.

Procedure:

- Implement the update rule $\theta = \theta - \eta \nabla J(\theta)$ for a chosen loss function J .
- Run gradient descent with learning rates $\eta \in \{0.001, 0.01, 0.1, 1, 5\}$.
- Try several different random initial values of θ for each learning rate.
- Record the loss versus iteration curve and the number of iterations needed to reach convergence (loss change below a small threshold).

Observations & Analysis:

Very small learning rates converge slowly, requiring many iterations to reach the minimum. A well-tuned learning rate converges quickly and smoothly. Too-large learning rates cause the loss to oscillate or diverge outright, overshooting the minimum at every step. Initialization mainly affects how many early iterations are needed, not the final optimum, since the loss surface here is convex.

Suggested Improvements:

- Momentum – accelerates convergence along consistent gradient directions and dampens oscillations across steep, narrow valleys in the loss surface.
- Adaptive learning-rate methods (Adagrad, RMSProp, Adam) – automatically adjust the effective learning rate per parameter, improving robustness to the initial learning-rate choice and speeding convergence, especially on ill-conditioned loss surfaces.

14. Design an experiment to implement a Neural Network classifier for a classification task. Vary the number of hidden layers, neurons, and activation functions. Evaluate the model using accuracy and loss curves, and analyze how network architecture affects performance, convergence, and overfitting.

Answer:

Objective:

To train a neural network classifier while varying the number of hidden layers, neurons per layer, and activation functions.

Procedure:

- Build MLP architectures with 1, 2, and 3 hidden layers.
- Vary the number of neurons per layer (e.g., 8, 32, 128).
- Compare ReLU, sigmoid, and tanh activation functions.
- Train each configuration for a fixed number of epochs, recording accuracy and loss curves on both training and validation sets.

Observations & Analysis:

Very small networks (few neurons/layers) underfit — both training and validation accuracy plateau at a low level. Deeper/wider networks fit the training data much better but, beyond a certain point, show a growing gap between training and validation performance (overfitting) unless regularized. ReLU activations converge noticeably faster and avoid vanishing gradients compared to sigmoid/tanh, especially as depth increases.

Conclusion:

Architecture choice directly controls the bias–variance trade-off — network capacity should be matched to the dataset's size and complexity, with regularization techniques (dropout, early stopping) becoming necessary as capacity increases.

15. Conduct an experiment using a hierarchical clustering algorithm on an unlabeled dataset. Compare different linkage methods such as single, complete, and average linkage. Evaluate clustering quality using dendrogram analysis and silhouette score. Analyze how linkage criteria influence cluster formation and interpretability of results.

Answer:

Objective:

To apply hierarchical clustering with single, complete, and average linkage on an unlabeled dataset and compare cluster quality.

Procedure:

- Compute pairwise distances between all data points.
- Build dendrograms using single, complete, and average linkage.
- Cut each dendrogram to form the same number of clusters for a fair comparison.
- Compute the silhouette score for each linkage method and visually inspect the dendrogram structure.

Observations & Analysis:

Single linkage tends to produce elongated, 'chained' clusters and is sensitive to noise/outliers (the chaining effect), often yielding a lower silhouette score. Complete linkage produces more compact, evenly-sized clusters. Average linkage is a balanced compromise between the two, usually producing dendrograms that are easier to interpret and cut at a meaningful level.

Conclusion:

Linkage criterion strongly shapes both cluster geometry and interpretability; average or complete linkage is generally preferred for well-separated, compact clusters, while single linkage is only appropriate when genuinely elongated or chain-like clusters are expected.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	15	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand